UTEC
Universidad Tecnológica

Patrón Prototype

El **Patrón Prototype** es un patrón de diseño creacional que permite crear nuevos objetos copiando o clonando una instancia existente, conocida como prototipo. Este patrón es útil cuando la creación de un nuevo objeto desde cero es costosa en términos de tiempo o recursos, y se prefiere crear una copia de un objeto ya existente, que luego puede ser modificado según sea necesario.

Objetivo

El objetivo principal del patrón Prototype es reducir el costo de creación de objetos al permitir la clonación de un objeto existente, en lugar de construir uno nuevo desde cero. Esto es especialmente útil en sistemas que requieren la creación repetitiva de objetos complejos.

Cómo Funciona

1. **Prototipo**: Es una interfaz que declara el método `clone()`, que es responsable de clonar objetos.
2. **Concrete Prototype**: Es una clase que implementa la interfaz Prototype y define el método `clone()` para crear una copia de sí misma.
3. **Cliente**: El cliente solicita una nueva instancia de un objeto a través de la clonación de un prototipo en lugar de crearlo desde cero.

Clonación Superficial vs. Clonación Profunda

- **Clonación Superficial (Shallow Copy)**: En la clonación superficial, se crea una nueva instancia del objeto, pero los objetos referenciados (como otros objetos o arrays) no se clonan. En su lugar, se copian las referencias a los objetos originales. Esto significa que los cambios en los objetos referenciados afectarán tanto al clon como al original.
- **Clonación Profunda (Deep Copy)**: En la clonación profunda, no solo se clona el objeto principal, sino también todos los objetos referenciados, de manera recursiva. De esta forma, el clon y el original son completamente independientes, y los cambios en uno no afectan al otro.

Ejemplo en Java

Supongamos que tenemos una clase **"Coche"** con varios atributos, incluidos algunos objetos referenciados, como un motor.

UTEC
Universidad Tecnológica

1. Definición de la clase Coche (con clonación superficial)

```
class Motor {
    private String tipo;

    public Motor(String tipo) {
        this.tipo = tipo;
    }

    public String getTipo() {
        return tipo;
    }

    public void setTipo(String tipo) {
        this.tipo = tipo;
    }
}

class Coche implements Cloneable {
    private String marca;
    private String modelo;
    private Motor motor;

    public Coche(String marca, String modelo, Motor motor) {
        this.marca = marca;
        this.modelo = modelo;
        this.motor = motor;
    }

    public void setMarca(String marca) {
        this.marca = marca;
    }

    public void setModelo(String modelo) {
        this.modelo = modelo;
    }

    public void setMotor(Motor motor) {
        this.motor = motor;
    }

    @Override
    protected Object clone() throws CloneNotSupportedException {
```

UTEC
Universidad Tecnológica

```
        return super.clone(); // Clonación superficial
    }

    @Override
    public String toString() {
        return "Coche [marca=" + marca + ", modelo=" + modelo + ",
motor=" + motor.getTipo() + "]";
    }
}
```

2. Uso de la clonación superficial

```
public class Main {
    public static void main(String[] args) {
        try {
            Motor motor = new Motor("V8");
            Coche cocheOriginal = new Coche("Ford", "Mustang", motor);

            Coche cocheClon = (Coche) cocheOriginal.clone();
            cocheClon.setMarca("Chevrolet");
            cocheClon.setModelo("Camaro");

            System.out.println("Coche Original: " + cocheOriginal);
            System.out.println("Coche Clon: " + cocheClon);

            // Modificando el motor del coche clonado
            cocheClon.getMotor().setTipo("V6");

            System.out.println("Después de modificar el motor del
clon:");
            System.out.println("Coche Original: " + cocheOriginal);
            System.out.println("Coche Clon: " + cocheClon);

        } catch (CloneNotSupportedException e) {
            e.printStackTrace();
        }
    }
}
```

Resultado:

```
Coche Original: Coche [marca=Ford, modelo=Mustang, motor=V8]
Coche Clon: Coche [marca=Chevrolet, modelo=Camaro, motor=V8]
Después de modificar el motor del clon:
Coche Original: Coche [marca=Ford, modelo=Mustang, motor=V6]
Coche Clon: Coche [marca=Chevrolet, modelo=Camaro, motor=V6]
```

Explicación: En la clonación superficial, aunque `cocheClon` y `cocheOriginal` son instancias diferentes, el motor sigue siendo el mismo objeto referenciado. Por lo tanto, cambiar el tipo de motor en el clon afecta también al objeto original.

3. Clonación Profunda en Java

Para implementar una clonación profunda, necesitamos modificar el método `clone()` para clonar también los objetos referenciados:

```java
@Override
protected Object clone() throws CloneNotSupportedException {
    Coche cocheClon = (Coche) super.clone();
    cocheClon.motor = new Motor(this.motor.getTipo()); // Clonación
profunda del motor
    return cocheClon;
}
```

Ahora, al clonar el coche, el motor también se clona, por lo que cualquier modificación en el motor del clon no afectará al original.

Resultado con Clonación Profunda:

```
Coche Original: Coche [marca=Ford, modelo=Mustang, motor=V8]

Coche Clon: Coche [marca=Chevrolet, modelo=Camaro, motor=V8]

Después de modificar el motor del clon:

Coche Original: Coche [marca=Ford, modelo=Mustang, motor=V8]

Coche Clon: Coche [marca=Chevrolet, modelo=Camaro, motor=V6]
```

Explicación: En este caso, el motor del clon y el del original son objetos diferentes, por lo que los cambios en uno no afectan al otro.

Ventajas del Patrón Prototype

1. **Optimización del rendimiento**: Es útil para reducir el costo de creación de objetos cuando es costoso crear objetos desde cero.
2. **Flexibilidad**: Permite la creación de nuevos objetos mediante la clonación de prototipos existentes, lo que facilita la creación de objetos personalizados.
3. **Independencia del sistema de creación**: El cliente no necesita conocer los detalles de cómo se crea un objeto; simplemente clona uno existente.

Conclusión

El patrón Prototype es una poderosa herramienta en situaciones donde la creación de objetos es costosa o cuando se desea una copia exacta de un objeto con modificaciones mínimas. Comprender la diferencia entre clonación superficial y profunda es crucial para implementar este patrón de manera efectiva en Java.